

Lab 3: KNN, Bayes Classifier and Decision Trees

T-504-ITML - Machine Learning / T-711-FOML - Fundamentals of Machine Learning

Fall, 2026

1 Introduction

The goal of this lab is to practice the workflow in supervised learning, including, preparing the data, splitting data into training and test set, finding the right parameters for training a model and evaluating the machine learning algorithm and the resulting model. As an example, we will use text data, in particular posts from different newsgroups and try create some classifiers that predicts the topic (newsgroup) of the text.

You will need scikit-learn, pandas and seaborn libraries. You can work in groups of up to three students (ideally 2).

2 Handing in

Hand in a **report** (PDF document or Jupyter notebook) with the answers to all the questions below including the respective Python code that generated the plot or numbers for your answer, if any. If you hand in a Jupyter notebook, make sure that it contains the output of all the code, so we don't have to run it ourselves.

3 Tasks

1. Loading and exploring data (*Total points: 15*)

The dataset we use contains posts to Usenet newsgroups in different groups. The task is to learn a model that can predict the topic of a text based on its content. We could use the data to train such a model by predicting the group name based on the content of the post.

Use the following code to load the dataset (this will download the dataset, the first time it is run).

```
from sklearn.datasets import fetch_20newsgroups
newsgroups = fetch_20newsgroups(subset='all')
```

- (a) (2 points) How many instances (examples) and variables are in the dataset?
- (b) (3 points) How many and which classes does the dataset define? What is their share in the data?
- (c) (5 points) Make a plot showing the class distribution in the dataset.
- (d) (5 points) Look at a few of the samples. Is there anything in the examples that might cause a classifier to overfit to the training data? How can you fix this? Change the code above accordingly! (You may want to check the documentation of `fetch_20newsgroups`.)

2. Splitting the data into training and test data (*Total points: 10*)

- (a) (5 points) Split the data into a training and test set using 20% of the data for testing, ensuring class distributions between the training and test data remain the same. (`train_test_split` might be useful.)
- (b) (5 points) Make a plot to check that the class distributions in the test set is the same as for the distribution in the whole data set.

3. Feature Engineering / Extraction (*Total points: 15*)

The data we are dealing with is basically unstructured text. In order to train a classifier we need to create features from the data to be used as the attributes of the samples. Scikit-learn provides us with some simple feature engineering / extraction for text. To start with we can use a simple Bag of Words representation, where each text is represented by a vector of integers, each one counting the occurrences of specific words in the text.

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X_features = vectorizer.fit_transform(X)
```

- (a) (5 points) How many attributes do we get from this feature engineering/extraction? Do you think this is a good number? Why (not)?
 - (b) (5 points) Change the parameters of CountVectorizer to exclude words appearing in more than half the instances and words appearing in less than 0.1% of the instances. How many attributes do you get after that change?
 - (c) (5 points) The output of CountVectorizer is not always very informative. Words that appear often (like 'it', 'is', 'has', etc) tend to have higher values, but contain little information about the topic because they appear in many different documents. We could try weight words higher if they appear in fewer documents. TfidfTransformer does exactly that. Apply a TfidfTransformer to the output of the CountVectorizer!
- ### 4. Training a model and evaluating with cross-validation (*Total points: 25*)
- (a) (5 points) Use scikit-learn's KNeighborsClassifier to learn a K-nearest Neighbor Classifier from the training data.
 - (b) (5 points) How many neighbors does the classifier use?
 - (c) (5 points) Use 5-fold cross-validation to estimate the models accuracy on both training and validation set. (It might be easiest to use sklearn.model_selection.cross_validate() for this).
 - (d) (5 points) What do the numbers tell you about overfitting/underfitting of the model?
 - (e) (5 points) Replace the KNN classifier with a Naive-Bayes classifier (MultinomialNB) and measure its accuracy using cross-validation. How well does this model perform in comparison?
- ### 5. Pipeline and Hyperparameter-Tuning (*Total points: 35*)
- Both preprocessing steps, as well as the classifier have hyper-parameters that could be tuned to get a better model. This is easiest to do, if we put all of these steps together in a pipeline that can be treated as an estimator without having to run each individual step when testing different parameter values.
- (a) (5 points) Pick a classifier (KNN Classifier, Naive-Bayes, decision tree, or something else that you tried before). Put all the preprocessing steps and the classifier into a pipeline.
 - (b) (10 points) Select 1 or 2 hyperparameters for each step in the pipeline that you find most important for the task. Select around 3 reasonable values for each hyper-parameter (2 in case of non-numeric hyperparameters that only have two options) and setup a search for the best combination of hyper-parameter values in the pipeline. This can be done for example using GridSearchCV or RandomizedSearchCV (see the user guide on grid search). Note: When setting this up, think about the potential run-time of the search. For each tested parameter combination, the search will do cross-validation. Think about how many combinations of values you want to test and how long it takes run the pipeline once before running this, otherwise you might be surprised about the run-time.
 - (c) (5 points) Report on the results. Which parameter combination is best? How good is it? Does that model still suffer from overfitting?
 - (d) (10 points) Analyze the results of the search in more detail. Which of the tuned parameters seems to have the most influence on performance of the model (reducing overfitting or underfitting)? Which parameters are independent/dependent on each other? (the independent ones could have been tuned separately)
 - (e) (5 points) Try out other preprocessing steps, other hyper-parameters and other models (e.g., TfidfVectorizer or MultinomialNB). What is the best model you can come up with?